

COLD STORAGE MANAGEMENT SYSTEM

A Mini Project Report submitted in the partial fulfillment of the

Requirements for the award of the degree

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

M.Sai Kumar (18471A0530)

N.Rajesh (18471A0535)

K.S.V.S.Chandra Prasad (18471A0521)

Under the esteemed guidance of

Dr.S.N.Tirumala Rao, M.Tech., Ph.D

Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

NARASARAOPETA

(Affiliated to JNTUK, Kakinada, Approved by AICTE & Thrice Accredited by NBA)

2020 - 2021

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the mini project that is entitled with the name **“COLD STORAGE MANAGEMENT SYSTEM”** is a bonafide work done by the team **M.Sai Kumar (18471A0530), N.Rajesh (18471A0535), K.S.V.S.Chandra Prasad (18471A0521)** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of **COMPUTER SCIENCE AND ENGINEERING** during 2020-2021.

PROJECT GUIDE

Dr. S. N. Tirumala Rao, M.Tech., Ph.D.,

Professor

PROJECT CO-ORDINATOR

T.Roopa, M.Tech,

Asst. Professor

HEAD OF THE DEPARTMENT

Dr. S. N. Tirumala Rao, M.Tech., Ph.D.,

Professor

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We wish to express my thanks to carious personalities who are responsible for the completion of the project. We are extremely thankful to our beloved chairman sri **M.V.Koteswara Rao,B.Sc.**, who took keen interest in us in every effort throughout this course. We owe out sincere gratitude to our beloved principal **Dr.M.Sreenivasa Kumar, M.Tech., Ph.D., MISTE., FIE(I).**, for showing his kind attention and valuable guidance throughout the course.

We express our deep felt gratitude towards **Dr.S.N.Tirumala Rao, M.Tech, Ph.D.** ,HOD of CSE department and also to our guide **Dr.S.N.Tirumala Rao, M.Tech, Ph.D** of CSE department whose valuable guidance and unstinting encouragement enable us to accomplish our project successfully in time.

We extend our sincere thanks towards **T.Roopu**, M.Tech Assistant professor & Project coordinator of the project for extending her encouragement. Their profound knowledge and willingness have been a constant source of inspiration for us throughout this projectwork.

We extend our sincere thanks to all other teaching and non-teaching staff to department for their cooperation and encouragement during our B.Tech degree.

We have no words to acknowledge the warm affection, constant inspiration and encouragement that we received from our parents.

We affectionately acknowledge the encouragement received from our friends and those who involved in giving valuable suggestions had clarifying out doubts which had really helped us in successfully completing our project.

By

M.Sai kumar	(18471A0530)
N.Rajesh	(18471A0535)
K.S.V.S.Chandra Prasad	(18471A0521)

ABSTRACT

India is one of the largest producers of agricultural products having diverse agro-climate zones across the country with vast range of agricultural products. The reprehensible storage results in high losses in grains, 30% of fruits and vegetables grown. During storage, quantitative as well as qualitative losses occur due to insects, rodents, and micro- organisms. To address the issues of the above concern and maintain proper stock details off all type of qualitative and qualitative loss products including their costs, we propose Cold Storage Management System (CSMS) for cold storage. Hence the product availability in cold storage with all necessary details of product are available by this mini project. Cold Storage Management System is a Web-based Application.

INDEX

S.No.	CONTENTS	PAGE NO
1	Introduction	1
2	System Analysis	2
	2.1 Literature Survey	2
	2.2 Existing System	2
	2.3 Proposed System	2
	2.4 System Requirements	3
3	System Design	4
	3.1 Module Description	4
	3.2 Database Table	5
	3.3 UML Diagrams	9
4	Implementation	15
5	Testing and Test Cases	27
	5.1 Introduction	27
	5.2 Testing Strategies	27
	5.3 Unit Testing	27
	5.4 Integration Testing	28
	5.5 Test Cases	29
6	Output Screens	33
7	Conclusion	38
8	Future Enhancements	39
9	Bibliography	40

List of Figures

S.No.	CONTENTS	PAGE NO
1	Fig : 3.3.1 : Activity Diagram	9
2	Fig : 3.3.2 : Usecase Diagram	10
3	Fig : 3.3.3 : Sequence Diagram	11
4	Fig : 3.3.4 : Class Diagram	13
5	Fig : 3.3.8 : ER Diagram	14
6	Fig : 5.1 : Customer Login Test Case 1	30
7	Fig : 5.2 : Customer Login Test Case 2	30
8	Fig : 5.3 : Customer Login Test Case 3	31
9	Fig : 5.4 : Customer Login Test Case 4	31
10	Fig : 5.5 : Customer Login Test Case 5	32
11	Fig : 5.6 : Customer Login Test Case 6	32
12	Fig : 6.1 : Login Page	33
13	Fig : 6.2 : Admin Page	33
14	Fig: 6.3 : Admin Control Page	34
15	Fig : 6.4 : Customer Entry Form For Admin	34
16	Fig : 6.5 : Incoming Product Entry Form For Admin	35
17	Fig: 6.6 : Outgoing Product Entry Form For Admin	35
18	Fig : 6.7 : Payment Details Entry Form For Admin	36
19	Fig : 6.8 : Customer Loan Entry Form For Admin	36
20	Fig : 6.9 : Summary form for admin	37

List of Tables

S.No.	CONTENTS	PAGE NO
1	Table: 3.2.1: Admin Table	5
2	Table: 3.2.2: Customer Table	5
3	Table: 3.2.3: Inward Products Table	6
4	Table: 3.2.4: Outward Products Table	6
5	Table: 3.2.5: Payment Table	7
6	Table: 3.2.6: Invoice Table	7
7	Table: 3.2.7: Customer Loan Table	8
8	Table: 5.1: Test Cases Data	29

1. INTRODUCTION

Cold storage is the one widely practiced method for bulk handling of the perishables between production and marketing processing. It is one of the methods of reserving perishable commodities in fresh and whole some state for a longer period by controlling temperature and humidity with in the storage system. Maintaining adequately low temperature is critical, as otherwise it will cause chilling injury to the produce. Also, relative humidity of the storeroom should be kept as high as 80-90% for most of the perishables, below (or) above which his detrimental effect on the keeping quality of the produce.

Most agro products have a very limited life after harvest if held at normal harvesting temperatures. Postharvest cooling rapidly removes field heat, allowing longer storage periods. Proper postharvest cooling can:

- Reduce respiratory activity and degradation by enzymes;
- Reduce internal water loss and wilting;
- Slow or inhibit the growth of decay-producing microorganisms;
- Reduce the production of the natural ripening agent, ethylene.

In addition to helping maintain quality, postharvest cooling also provides marketing flexibility by allowing the grower to sell produce at the most appropriate time. Having cooling and storage facilities makes it unnecessary to market the produce immediately after harvest. This can be an advantage to growers who supply restaurants and grocery stores or to small growers who want to assemble truckload lots for shipment. Postharvest cooling is essential to delivering produce of the highest possible quality to the consumer.

The cold storage of perishables has advanced noticeably in recent years, leading to better maintenance of organoleptic qualities, reduced spoilage, and longer shelf lives.

2. SYSTEM ANALYSIS

2.1 LITERATURE SURVEY

India is one of the largest producers of agricultural products having diverse agro-climate zones across the country with vast range of agricultural products. The reprehensible storage results in high losses in grains, 30% of fruits and vegetables grown. During storage, quantitative as well as qualitative losses occur due to insects, rodents, and micro- organisms.

2.2 EXISTING SYSTEM

The Cold Store Management System is designed to be used where:-

- It is necessary to control stock both by pallet (or carton or container) and by stock type where the stock location needs to be controlled by the system.
- It is important to be able to control stock by receipt date / date of manufacture / the sell by date.
- This means that the oldest useable stock will always be issued first irrespective of its location in the Cold Store.
- Stock is stored on behalf of third parties i.e. commercial Cold Stores.
- This is all done with maintaining records manually in a log book or in a MS excel to generate bills and maintain the stock details.

2.3 PROPOSED SYSTEM

- All software projects start by identifying a clear objective or goal.
- Major objective of this report is to analysis Design and Implementation of Cold storage Management System.
- Looking at the problems posed by the existing system of Cold storage, the objective can be sum up as follows to develop a Cold storage Management System for efficiently and effectively store all information related to different payments, loan, operational advance etc.
- Allow the Management to easily access that information as necessary.
- Less Documentation and Faster working performance and thus saving times.
- In a form the information is desired by the Management, in a quick and effective order so that decisions can be made for proper and optimal business operations.

2.4 SYSTEM REQUIREMENTS

2.4.1 HARDWARE REQUIREMENTS

- ❖ Processor : Intel ® Dual Core 2.0GHz minimum
- ❖ Hard Disk : 80GB minimum
- ❖ RAM : 4GB or More

2.4.2 SOFTWARE REQUIREMENTS

- ❖ Operating System : Windows 2012 Server or later
- ❖ Language : .NET Framework 4.5
- ❖ Database Server : SQL server 2012 or later
- ❖ Web Server : IIS 8.0
- ❖ Browser : Any latest browser like Chrome

3 SYSTEM DESIGN

3.1 MODULE DESCRIPTION

The system consists of two modules. They are:

1. Administrator Module
2. Customer Module

3.1.1 ADMINISTRATOR MODULE

- An Admin can login in to the his Admin Home Page by typing Customer name and password, and he can create, update, and delete a customers, inward products ,outward products, payment details, invoice ,customer loan details .
- He can manage the website and give new Queries. And he can add customers, inward products, outward products, payment details, invoice, customer loan details.

3.1.2 CUSTOMER MODULE

- Customers are required to give valid Customer details such as customer id and password to get their invoice.

3.2 DATABASE TABLES

Admin Details Table

S.No	Field	Type	Constraint	Description
1	userID	varchar(50)	Primary Key	Login ID for Admin
2	userPassword	varchar(50)	Not Null	Password for Login

Table: 3.2.1 Admin Table

Customer Details Table

S.No	Field	Type	Constraint	Description
1	customerID	varchar(50)	Primary Key	Customer ID
2	customerName	varchar(50)	Not Null	Customer Name
3	customerPhoneNumber	Number(10)	Not Null	Customer Phone Number
4	customerEmail	varchar(50)	Not Null	Customer Email ID
5	customerHouseNumber	varchar(50)	Not Null	Customer House Number
6	customerStreetName	varchar(50)	Not Null	Customer Street Name
7	customerCity	varchar(50)	Not Null	Customer City
8	customerZipCode	Number(10)	Not Null	Customer Zip Code
9	customerState	varchar(50)	Not Null	Customer State

Table: 3.2.2 Customer Table

Inward Product Details Table

S.No	Field	Type	Constraint	Description
1.	inwardProductID	varchar(50)	Primary Key	Incoming Product ID
2.	inProdDate	Date	Primary Key	Incoming Product Date
3.	inProdName	varchar(50)	Not Null	Incoming Product Name
4.	inProdCost	Number(10)	Not Null	Incoming Product Cost
5.	inProdQuantity	Number(10)	Not Null	Incoming Product Quantity
6.	inProdLocation	varchar(50)	Not Null	Incoming Product location
7.	customerID	varchar(50)	Foreign Key	Customer ID
8.	Insurance	varchar(50)	Not Null	Incoming Product Insurance
9.	inwardProductRent	Number(10)	Not Null	Incoming Product Rent

Table: 3.2.3 Inward Products Table

Outward Product Details Table

S.No	Field	Type	Constraint	Description
1.	outwardProductID	varchar(50)	Primary Key	Outgoing Product ID
2.	outwardProductDate	Date	Primary Key	Outgoing Product Date
3.	outwardProductName	varchar(50)	Not Null	Outgoing Product Name
4.	outwardProductCost	Number(10)	Not Null	Outgoing Product Cost
5.	outwardProductQuantity	Number(10)	Not Null	Outgoing Product Quantity
6.	outwardProductLocation	varchar(50)	Not Null	Outgoing Product Location
7.	outwardProductCustomerID	varchar(50)	Foreign Key	Customer ID

Table: 3.2.4 Outward Products Table

Payment Details

S.No	Field	Type	Constraint	Description
1.	paymentID	varchar(50)	Primary Key	Payment ID
3.	paymentType	varchar(50)	Not Null	Payment Type
4.	paymentAmount	money	Not Null	Payment Amount
5.	paymentDate	Date	Not Null	Payment Date
6.	paymentTime	Time	Not Null	Payment Time
7.	customerID	int	Foreign Key	Customer ID
8.	paymentAdvance	money	Null	Amount paid in advance
9.	paymentDue	money	Null	Amount to be paid

Table: 3.2.5 Payment Table

Invoice Details

S.No	Field	Type	Constraint	Description
1.	billNumber	varchar(50)	Primary Key	Invoice ID
2.	customerID	varchar(50)	Not Null	Customer ID
3.	outwardProductID	varchar(50)	Foreign Key	Outgoing Product ID
4.	paymentID	varchar(50)	Foreign Key	Payment ID
5.	outwardProductQuantity	Number(10)	Not Null	Outgoing Product Quantity
8.	Date	Date	Not Null	Invoice Generated Date
9.	Time	Time	Not Null	Invoice Generated Time

Table: 3.2.6 Invoice Table

Customer Loan Details

S.No	Field	Type	Constraint	Description
1	loanId	int	Primary Key	Customer Loan ID
2	loanDate	Date	Not Null	Loan sanctioned date
3	inwardProductID	int	Foreign Key	Incoming product ID
4	loanAmount	money	Not Null	Loan sanctioned amount
5	bankName	varchar(50)	Not Null	Loan sanctioned Bank name
6	customerAccount Number	bigint	Not Null	Customer account number
7	bankIFSCCode	varchar(50)	Not Null	Bank IFSC code

Table: 3.2.7 Customer Loan Table

3.3 UML DIAGRAMS

3.3.1 ACTIVITY DIAGRAM

An Activity diagram shows the flow from one activity to other activity within a system. Activity diagrams illustrate the dynamic view of a system. Activity diagrams are especially important in modelling the function of a system.

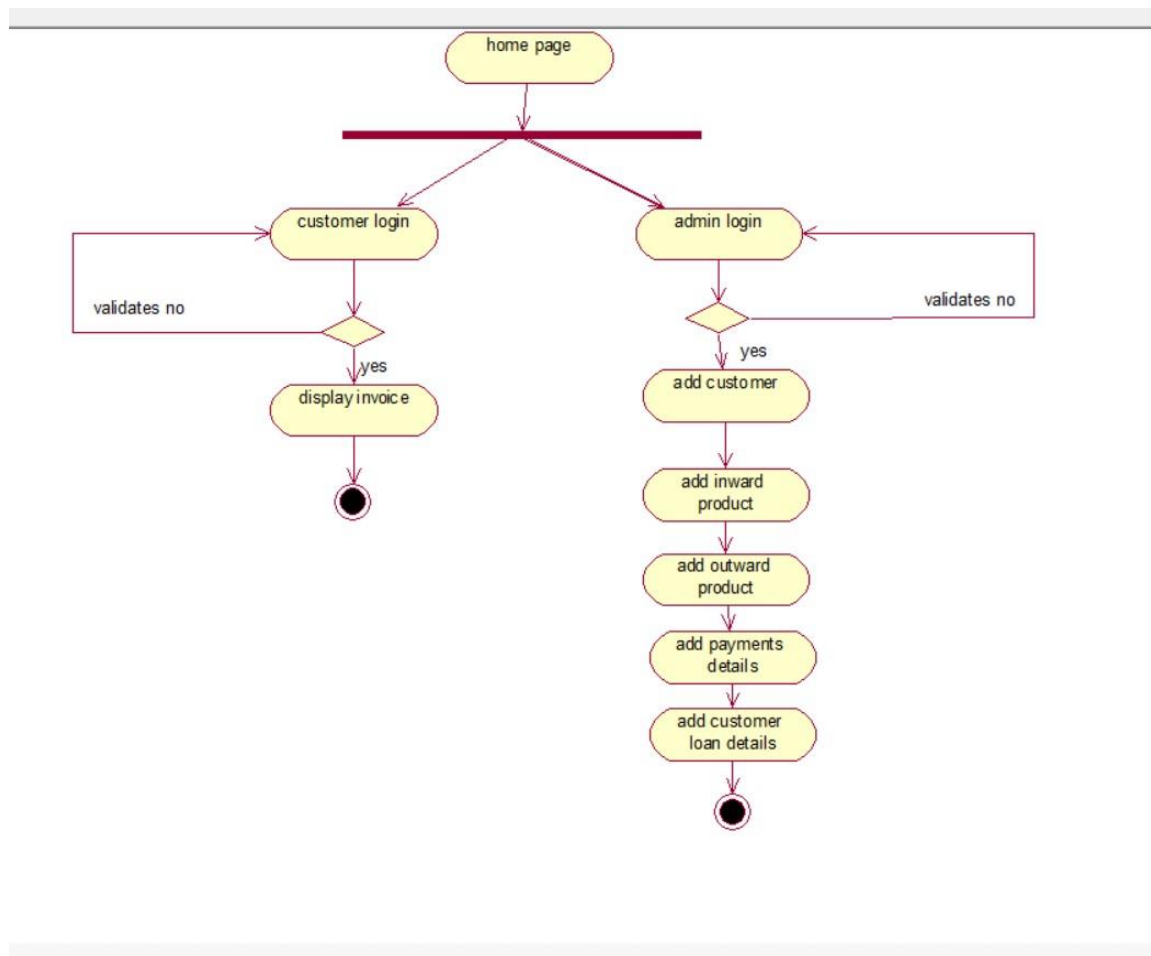


Fig: 3.3.1 Activity Diagram

3.3.2 USE CASE DIAGRAM

A Use Case Diagram shows a set of use cases, actors and their relationships. Use case diagrams illustrate the static use case view of a system. Use case diagrams are especially important in organizing and modelling the behavior of a system.

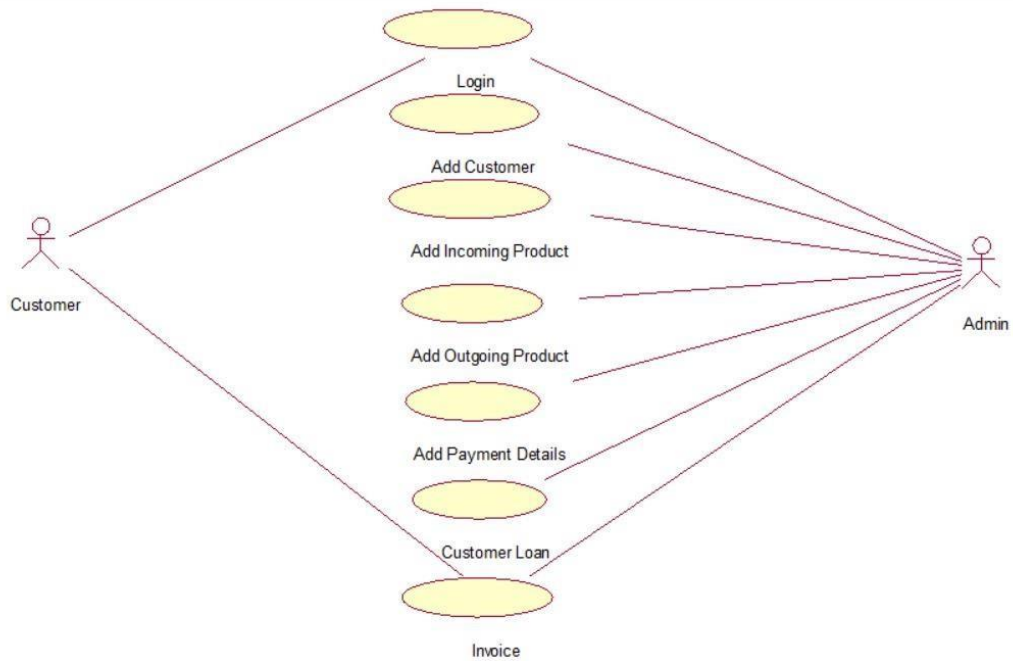


Fig: 3.3.2 Use Case Diagram

3.3.3 SEQUENCE DIAGRAM

A Sequence diagram is an interaction diagram that emphasizes the time ordering of messages. A sequence diagram shows a set of objects and the messages sent and received by those objects. Sequence diagrams illustrate the dynamic view of a system.

3.3.3.1 ADMIN

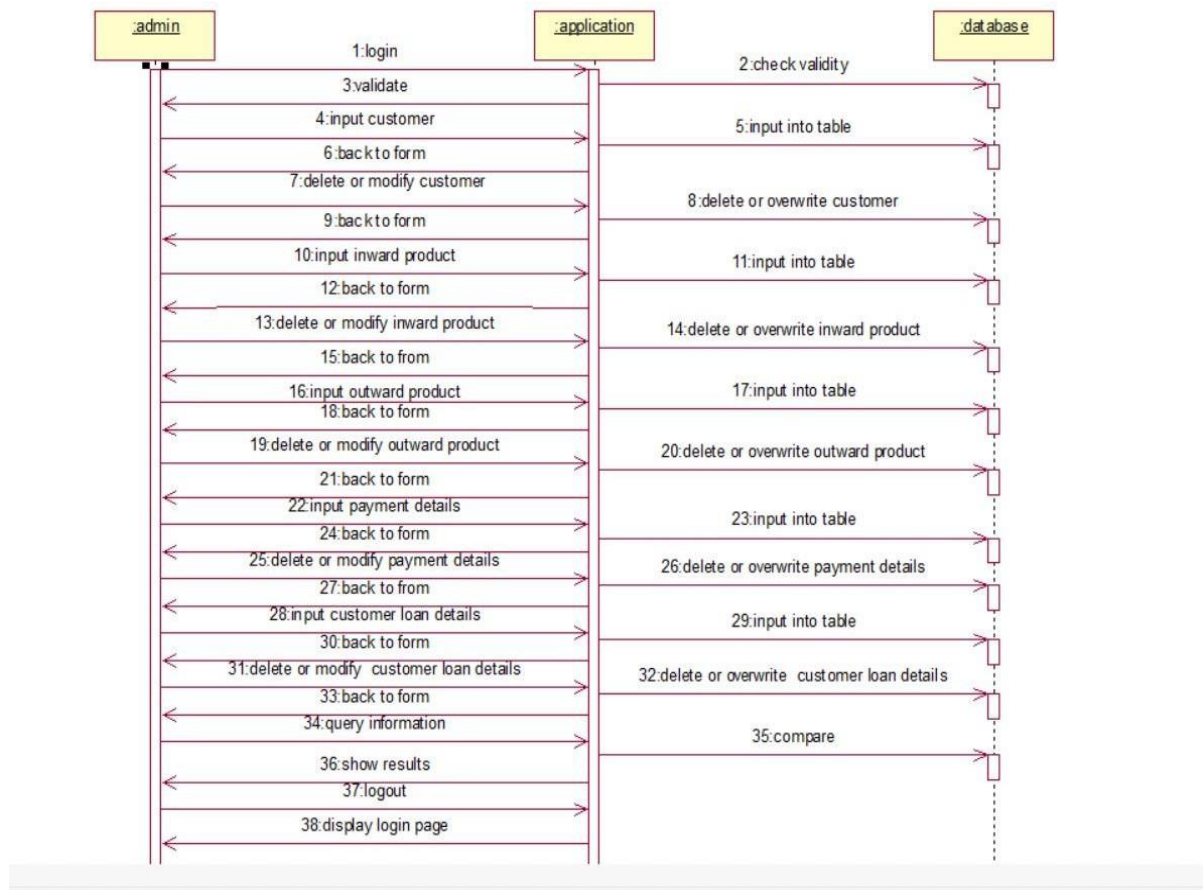


Fig:3.3.3.1 Admin Sequence Diagram

3.3.3.2 CUSTOMER

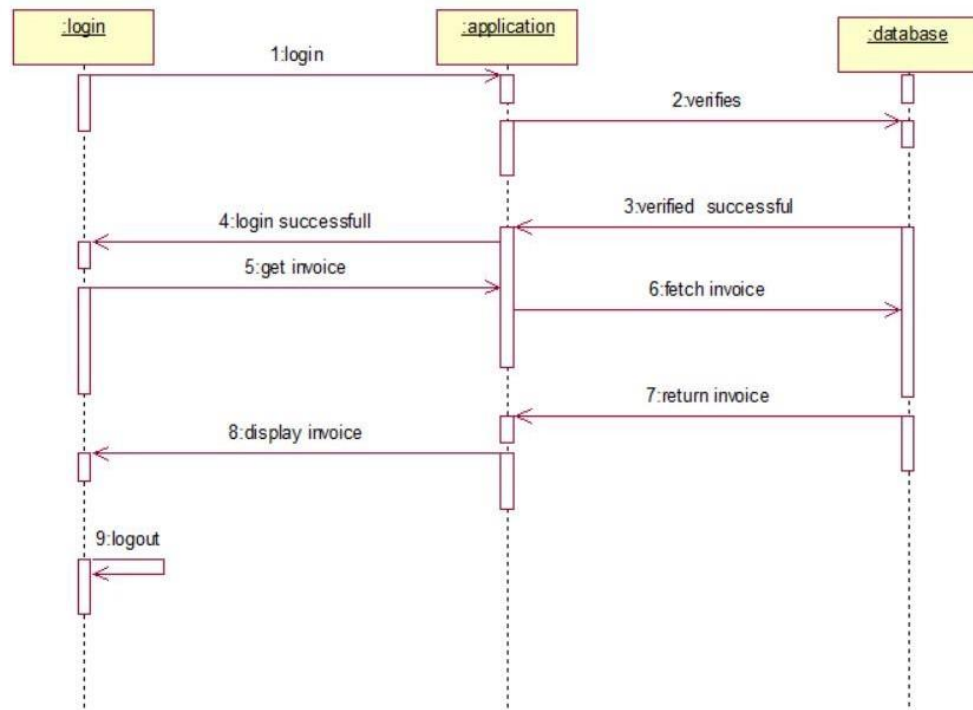


Fig: 3.3.3.2 Customer Sequence Diagram

3.3.4 CLASS DIAGRAM

Class diagram shows a set of classes, interface, collaborations and their relationships. Class diagrams are most common diagram found in modeling object oriented systems. Class diagrams illustrate the static view of a system. Class diagrams that include active classes are used to address the static process view.

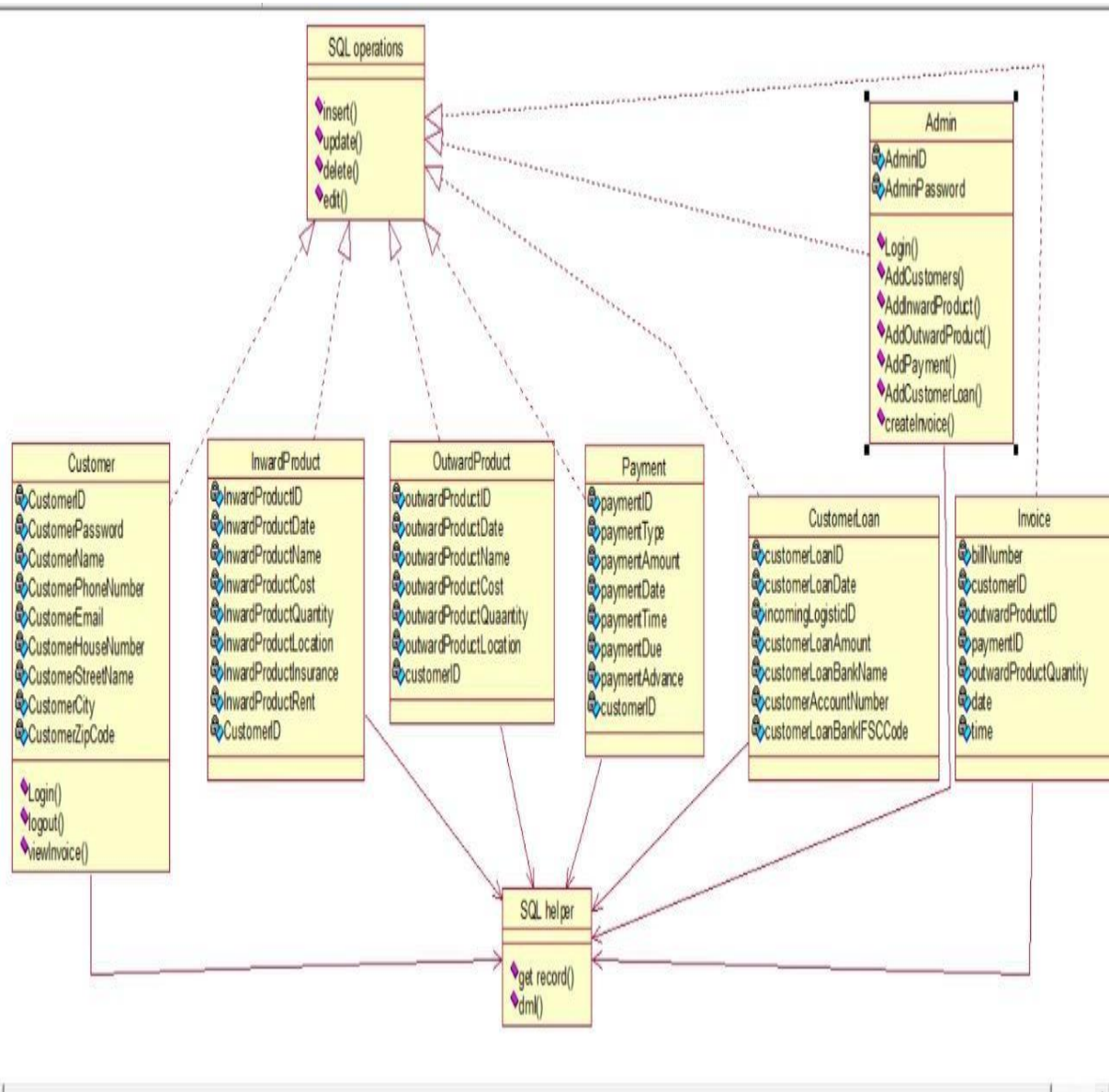


Fig:3.3.4 Class Diagram

3.3.5 ER DIAGRAM

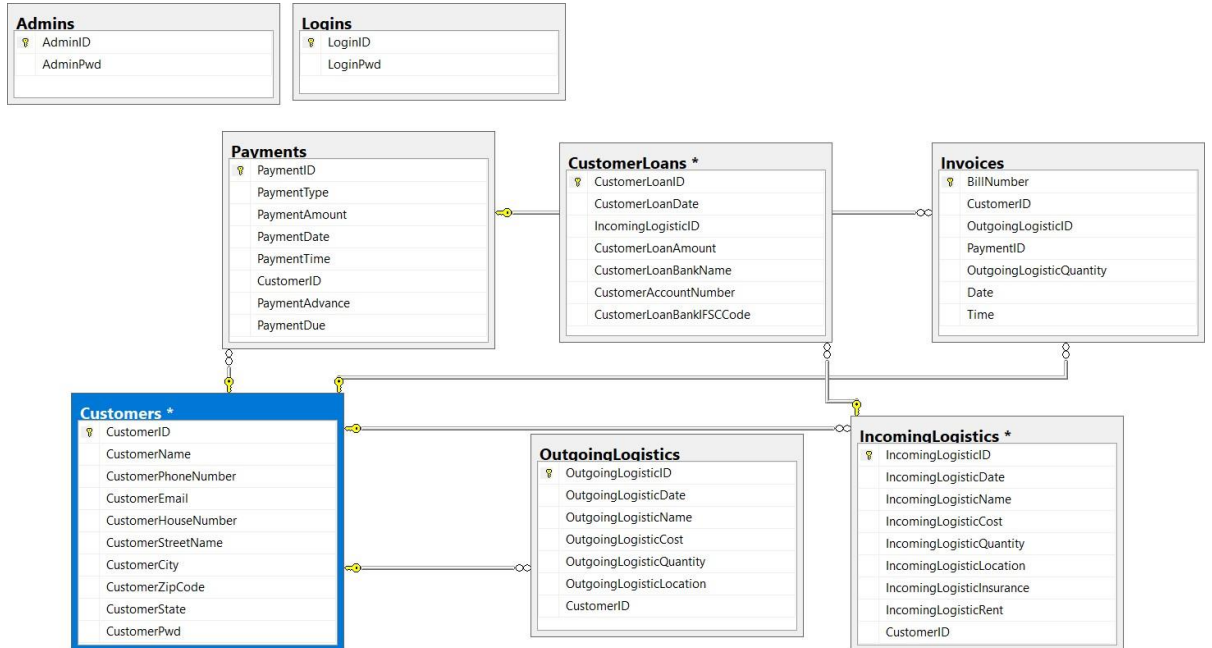


Fig:3.3.5 ER Diagram

4 IMPLEMENTATION

Sample Code:

Layout.cshtml:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@ViewBag.Title - My ASP.NET Application</title>
    @Styles.Render("~/Content/css")
    @Scripts.Render("~/bundles/modernizr")
</head>
<body>
    <div class="navbar navbar-inverse navbar-fixed-top">
        <div class="container">
            <div class="navbar-header">
                <button type="button" class="navbar-toggle" data-toggle="collapse" data-
target=".navbar-collapse">
                    <span class="icon-bar"></span>
                    <span class="icon-bar"></span>
                    <span class="icon-bar"></span>
                </button>
                @Html.ActionLink("Application name", "Index", "Home", null, new { @class =
"navbar-brand" })
            </div> <div class="navbar-collapse collapse">
```

```

        <ul class="nav navbar-nav">
            <li>@Html.ActionLink("Home", "Index", "Home")<br /></li>
            <li>@Html.ActionLink("About", "About", "Home")<br /></li>
            <li>@Html.ActionLink("Contact", "Contact", "Home")<br /></li>
            <li>@Html.ActionLink("Customer Login", "Login", "Home") <br /></li>
            <li>@Html.ActionLink("Admin Login", "Admin", "Home")<br /></li>
        </ul></div></div></div>

        <div class="container body-content">
            @RenderBody()

            <hr />

            <footer>

                <p>&copy; @DateTime.Now.Year - My ASP.NET Application</p>

            </footer></div>

            @Scripts.Render("~/bundles/jquery")
            @Scripts.Render("~/bundles/bootstrap")
            @RenderSection("scripts", required: false)
        </body>
    </html>

```

HomeController.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using MyFirstProj.Models;
using System.Data.SqlClient;
namespace MyFirstProj.Controllers
{

```

```

public class HomeController : Controller
{
    public ActionResult Index()
    {return View();}

    public ActionResult About()
    {
        ViewBag.Message = "";

        return View();
    }

    public ActionResult Contact()
    {
        ViewBag.Message = "Sai Kumar Mamidala";
        return View();
    }

    public ActionResult Login()
    {
        ViewBag.Message = "";
        return View();
    }

    [HttpPost]public ActionResult Login(Models.Login ObjLogin)
    {
        using(Models.dbContext context=new Models.dbContext())
        {
            var obj=context.Logins.Where(a=> a.LoginID == ObjLogin.LoginID &&
a.LoginPwd==ObjLogin.LoginPwd).FirstOrDefault();

            var x = context.Logins.Where(b => b.LoginID == ObjLogin.LoginID &&
b.LoginPwd != ObjLogin.LoginPwd).FirstOrDefault();

```

```

        var y = context.Logins.Where(a => a.LoginID != ObjLogin.LoginID&& a.LoginPwd
== ObjLogin.LoginPwd).FirstOrDefault();
        if(obj!= null)
        {
            String sql1 = "select * from Customers where CustomerID =" + Session["LoginID"]
+ "";
            context.Database.ExecuteSqlCommand(sql1);
            ViewBag.bio = sql1;
            Session["LoginID"] = ObjLogin.LoginID;
            return RedirectToAction("Invoice","Home");
        }else{
            if (x != null){
                ModelState.AddModelError("", "CUSTOMER PASSWORD provided is
INVALID");
            }else if (y != null){
                ModelState.AddModelError("", "CUSTOMER ID provided is INVALID");
            }else{
                ModelState.AddModelError("", " CUSTOMER USER ID/PASSWORD provided
is INVALID");
            }return View();
        }
    }
}

public ActionResult Admin(){
    ViewBag.Message = "";
    return View();
}

[HttpPost]public ActionResult Admin(models.Admin ObjAdmin){
    using (Models.dbContext con = new Models.dbContext())
    {

```

```

        var obj = con.Admins.Where(a => a.AdminID == ObjAdmin.AdminID &&
a.AdminPwd == ObjAdmin.AdminPwd).FirstOrDefault();

        var x = con.Admins.Where(b => b.AdminID == ObjAdmin.AdminID &&
b.AdminPwd != ObjAdmin.AdminPwd).FirstOrDefault();

        var y = con.Admins.Where(a => a.AdminID != ObjAdmin.AdminID &&
a.AdminPwd == ObjAdmin.AdminPwd).FirstOrDefault();

        if (obj != null)
        {
            Session["AdminID"] = ObjAdmin.AdminID;
            return RedirectToAction("Empty", "Home");
        }else{
            if (x != null){
                ModelState.AddModelError("", "PASSWORD provided is INVALID");
            } else if (y != null){
                ModelState.AddModelError("", "ADMIN ID provided is INVALID");
            } else{
                ModelState.AddModelError("", " ADMIN USER ID/PASSWORD provided is INVALID");
            }return View();
        }
    }

    public ActionResult logsuccess(){
        ViewBag.Message = "";
        return View();
    }

    public ActionResult Empty(){
        ViewBag.Message = "";
        return View();
    }
}

```

```

public ActionResult Summary(){
    ViewBag.CustomerDetails = getcustomer();
    ViewBag.OutgoingLogisticDetails = getOutgoingLogistic();
    ViewBag.PaymentDetails = getPayment();
    return View();
}

dbContext cd = new dbContext();

public List<Customer> getcustomer(){
    List<Customer> Cust = cd.Customers.ToList();
    return Cust;
}

public List<OutgoingLogistic> getOutgoingLogistic(){
    List<OutgoingLogistic> OutLog = cd.OutgoingLogistics.ToList();
    return OutLog;
}

public List<Payment> getPayment(){
    List<Payment> Pay = cd.Payments.ToList();
    return Pay;
}

public ActionResult CreatedSuccesfully(){
    ViewBag.Message = "";
    return View();
}

public ActionResult Invoice(){
    dbContext db = new dbContext();
    ViewBag.Message = "";
    string sql1 = "select * from Invoices where CustomerID='" + Session["LoginID"] + "'";
    db.Database.ExecuteSqlCommand(sql1);
}

```

```

        db.SaveChanges();
        return View();
    }
    private object List(){
        throw new NotImplementedException();
    }
}

```

CustomersController.cs

```

using System;
using System.Collections.Generic;
using System.Data;
using System.Data.Entity;
using System.Linq;
using System.Net;
using System.Web;
using System.Web.Mvc;
using MyFirstProj.Models;
using System.Data.SqlClient;

namespace MyFirstProj.Controllers{
    public class CustomersController : Controller{
        private DbContext db = new DbContext();

        public ActionResult Index(string searchName){
            var projects = from pr in db.Customers select pr;
            if (!String.IsNullOrEmpty(searchName)){
                projects = projects.Where(c => c.CustomerName.Contains(searchName));
            }
            return View(projects);
        }
    }
}

```

```

public ActionResult Details(int? id){
    if (id == null){
        return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    }
    Customer customer = db.Customers.SingleOrDefault(m => (m.CustomerID == id));
    if (customer == null){
        return HttpNotFound();
    }
    return View(customer);
}

public ActionResult Create(){
    return View();
}

[HttpPost] [ValidateAntiForgeryToken]public ActionResult
Create([Bind(Include="CustomerID,CustomerName,CustomerPhoneNumber,CustomerEmail,C
ustomerHouseNumber,CustomerStreetName,CustomerCity,CustomerZipCode,CustomerState")
] Customer customer){
    if (ModelState.IsValid){
        db.Customers.Add(customer);
        db.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(customer);
}

public ActionResult Edit(int? id){
    if (id == null){
        return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    }
    Customer customer = db.Customers.Find(id);
    if (customer == null){

```

```

        return HttpNotFound();
    }
    return View(customer);
}

[HttpPost] [ValidateAntiForgeryToken]public ActionResult
Edit([Bind(Include="CustomerID,CustomerName,CustomerPhoneNumber,CustomerEmail,Cus
tomerHouseNumber,CustomerStreetName,CustomerCity,CustomerZipCode,CustomerState")]
Customer customer){
    if (ModelState.IsValid){
        db.Entry(customer).State = System.Data.Entity.EntityState.Modified;
        db.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(customer);
}

public ActionResult Delete(int? id){
    if (id == null){
        return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    }
    Customer customer = db.Customers.Find(id);
    if (customer == null){
        return HttpNotFound();
    }
    return View(customer);
}

[HttpPost, ActionName("Delete")] [ValidateAntiForgeryToken]
public ActionResult DeleteConfirmed(int id){
    Customer customer = db.Customers.Find(id);
    db.Customers.Remove(customer);

```

```

        db.SaveChanges();
        return RedirectToAction("Index");
    }
    protected override void Dispose(bool disposing){
        if (disposing){
            db.Dispose();
        }
        base.Dispose(disposing);
    }
    public ActionResult Invoice(){
        ViewBag.Message = "";
        string sql1 = "select * from Invoices where CustomerID='" + Session["CustomerID"]
+""";
        return View(db.Invoices.SqlQuery(sql1).FirstOrDefault());
    }
}

```

dbContext.cs

```

using System;

using System.Collections.Generic;
using System.Data.Entity;
using System.Linq;
using System.Web;

namespace MyFirstProj.Models{
    public class dbContext : DbContext{
        public dbContext() : base("name=dbContext"){ }
        public System.Data.Entity.DbSet<MyFirstProj.Models.Customer> Customers { get; set; }
    }

    public System.Data.Entity.DbSet<MyFirstProj.Models.Login> Logins { get; set; }
    public System.Data.Entity.DbSet<MyFirstProj.Models.Admin> Admins { get; set; }
}

```

```

        public System.Data.Entity.DbSet<MyFirstProj.Models.IncomingLogistic>
IncomingLogistics { get; set; }

        public System.Data.Entity.DbSet<MyFirstProj.Models.OutgoingLogistic>
OutgoingLogistics { get; set; }

        public System.Data.Entity.DbSet<MyFirstProj.Models.CustomerLoan> CustomerLoans
{ get; set; }

        public System.Data.Entity.DbSet<MyFirstProj.Models.Payment> Payments { get; set; }

        public System.Data.Entity.DbSet<MyFirstProj.Models.Invoice> Invoices { get; set; }

    }}

```

IncomingDetails.cs

```

using System;

using System.Collections.Generic;

using System.ComponentModel.DataAnnotations;

using System.Linq;

using System.Web;

namespace MyFirstProj.Models{

    public class IncomingLogistic{

        public int IncomingLogisticID { get; set; }

        [DataType(DataType.Date)]

        [DisplayFormat(DataFormatString = "{0:MM/dd/yyyy}", ApplyFormatInEditMode =
true)]

        public DateTime IncomingLogisticDate { get; set; }

        public string IncomingLogisticName { get; set; }

        public decimal IncomingLogisticCost { get; set; }

        public Int64 IncomingLogisticQuantity { get; set; }

        public string IncomingLogisticLocation { get; set; }

        public Boolean IncomingLogisticInsurance { get; set; }

        public decimal IncomingLogisticRent { get; set; }

        public int CustomerID { get; set; }
    }
}

```

```

        public Customer Customer { get; set; }
    }
}

```

Invoice.cs

```

using System;

using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Web;

namespace MyFirstProj.Models{
    public class Invoice{
        [Key]
        public string BillNumber { get; set; }
        public int CustomerID { get; set; }
        public Customer Customer { get; set; }
        public int OutgoingLogisticID { get; set; }
        public OutgoingLogistic OutgoingLogistic { get; set; }
        public int PaymentID { get; set; }
        public Payment Payment { get; set; }
        public Int64 OutgoingLogisticQuantity { get; set; }

        [DataType(DataType.Date)] [DisplayFormat(DataFormatString = "{0:MM/dd/yyyy}",
        ApplyFormatInEditMode = true)]
        public DateTime Date { get; set; }

        [DataType(DataType.Time)] [DisplayFormat(DataFormatString = "{0:hh\\:mm}",
        ApplyFormatInEditMode = true)]
        public TimeSpan Time { get; set; }
    }
}

```

5. TESTING AND TEST CASES

5.1 INTRODUCTION

One of the purposes of the testing is to validate and verify the system. Verification means checking the system to ensure that it is doing what the function is supposed to do and Validation means checking to ensure that system is doing what the Customer wants it to do.

No program or system design is perfect; communication between the Customer and the designer is not always complete or clear, and time is usually short. The result is errors and more errors. Theoretically, a newly designed system should have all the pieces in working order, but in reality, each piece works independently. Now is the time to put all the pieces into one system and test it to determine whether it meets the Customer's requirements. This is the best chance to detect and correct errors before the system is implemented. The purpose of system testing is to consider all the likely variations to which it will be subjected and then push the system to its limits. If we implement the system without proper testing then it might cause the problems.

1. Communication between the Customer and the designer.
2. The programmer's ability to generate a code that reflects exactly the system specification.

The time frame for the design theoretically, a new designed system should have all the pieces in working order, but in reality, each piece works independently. Now is the time to put all the pieces into one system and test it to determine whether it meets the requirements of the Customer. The process of system testing and the steps taken to validate and prepare a system for final implementation are:

5.2 TESTING STRATEGIES

In order to make sure that the system does not have errors, the different levels of testing strategies that are applied at differing phases of software development are:

5.3 UNIT TESTING

It involves design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. It is the testing of individual software units of the application .It is done after the completion of an individual unit before integration.

5.4 INTEGRATION TESTING

In Integration Testing, the different units of the system are integrated together to form the complete system and this type of testing checks the system as whole to ensure that it is doing what is supposed to do. The testing of an integrated system can be carried out top-down, bottom-up, or big-bang. In this type of testing, some parts will be tested with white box testing and some with black box testing techniques. This type of testing plays very important role in increasing the systems productivity. We have checked our system by using the integration testing techniques.

VERIFICATION TESTING

In Verification Testing, the different units of system are verified together whether the exact output to the system is existing or not.

VALIDATION TESTING

In Validation Testing software validation is achieved through a series of Black box tests that demonstrate conformity with the requirements. After each validation test case has been conducted one or two possible conditions exists:

1. The function or performance characteristics conform to specification and are accepted.
2. A deviation from specification is uncovered and a deficiency list is created.

5.5 SYSTEM TESTING

Apart from testing the system to validate the functionality of software against the requirements, it is also necessary to test the non-functional aspect of the system. Some examples of non-functional tools include tests to check performance, data security, usability/Customer friendliness, volume, load/stress that we have used in our project to test the various modules. System testing consists of the following steps:

1. Program(s) testing.
2. String testing.
3. System testing.
4. System documentation.
5. Customer acceptance testing.

Input	Expected behaviour	Observed behaviour	Status
valid data for Customer Login	Login Completed Successfully	Login Completed Successfully	Success
Invalid Customer ID and valid password	Please enter valid Customer ID	Please enter valid Customer ID	Success
valid Customer ID and invalid password	valid Customer ID and invalid password	Please enter valid password	Success
invalid Customer ID and password	invalid Customer ID and password	Please enter valid password	Success
No Customer ID and password	No Customer ID and password	Please enter Customer ID and password	Success
valid data for Admin Login	Navigated to the page where admin maintains data	Navigated to the page where admin maintains data	Success
Invalid Admin ID and valid password	Please enter valid Admin ID	Please enter valid Admin ID	Success
valid Admin ID and invalid password	valid Admin ID and invalid password	Please enter valid password	Success
invalid Admin ID and password	invalid Admin ID and password	invalid Admin ID and password	Success
No Admin ID and password	No Admin ID and password	Please enter Admin	Success

Table: 5.1 Test cases data

Testcase 1: Customer login

Scenario 1:

Input : valid Customer name and password

Expected behaviour : Successful login

Actual behaviour: Successful login

Status: Pass

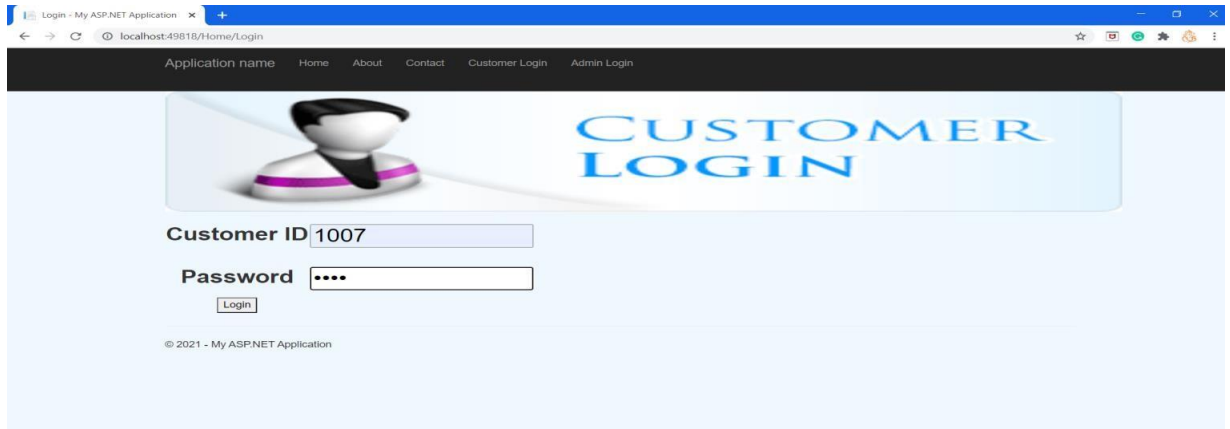


Fig: 5.1 Customer login test case 1

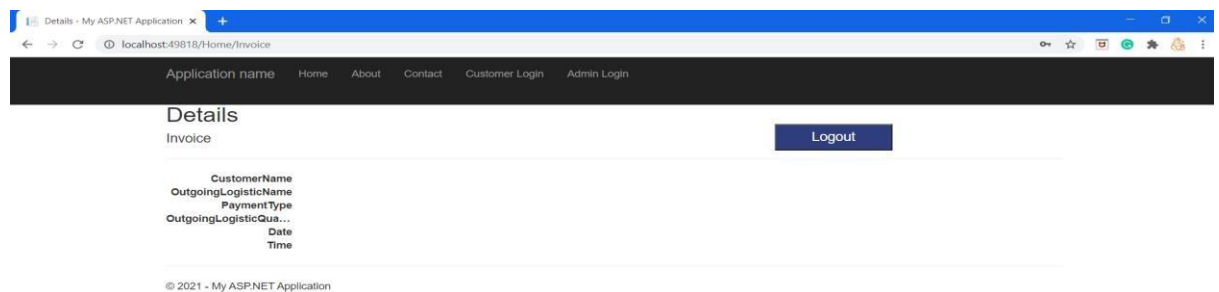


Fig: 5.2 Customer login test case 2

Scenario 2:

Input : Invalid Customer ID and valid password

Expected behaviour: Please enter valid Customer ID

Actual behaviour: Enter valid Customer ID

Status: Pass

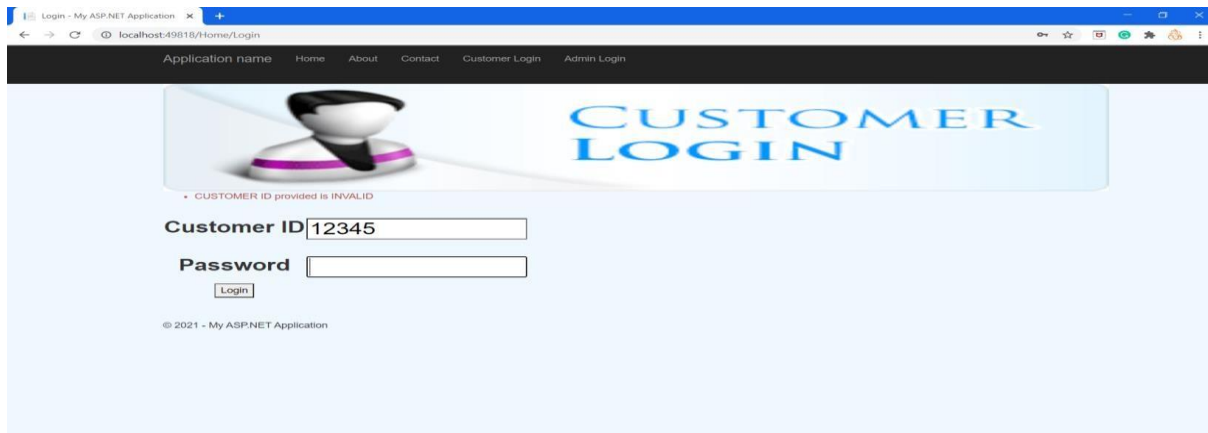


Fig: 5.3 Customer login Test case 3

Scenario 3: valid Customer name and invalid password

Input: valid Customer name and invalid password

Expected behaviour: Please enter valid password

Actual behaviour: Enter valid password

Status : Pass

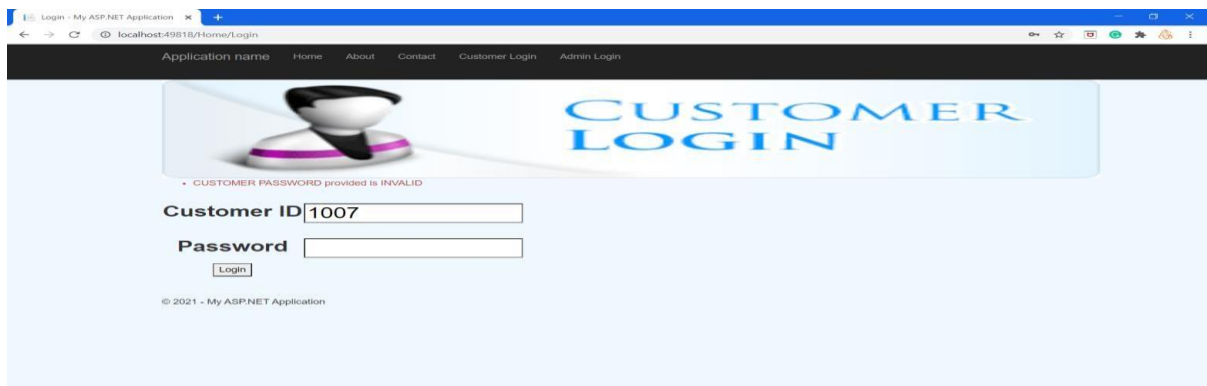


Fig: 5.4 Customer login Test case 4

Scenario 4: invalid Customer name and password

Input: invalid Customer name and password

Expected behaviour: Please enter valid Customer name and password

Actual behaviour: Please enter valid Customer name and password

Status: fail

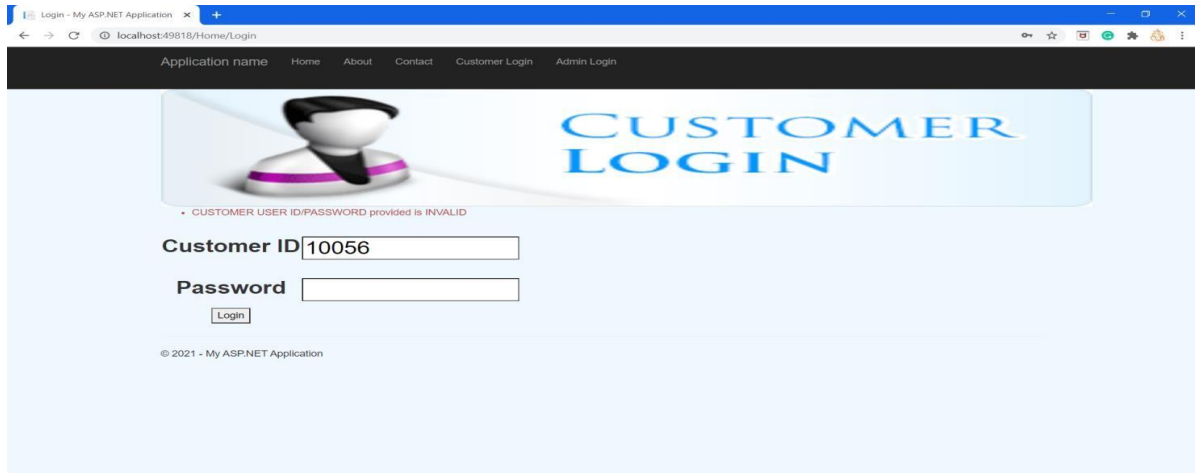


Fig: 5.5 Customer login Test case 5

Scenario 5: no Customer name and password

Input: no Customer name and password

Expected behaviour: Please enter Customer name and password

Actual behaviour: Please enter Customer name and password

Status: Fail

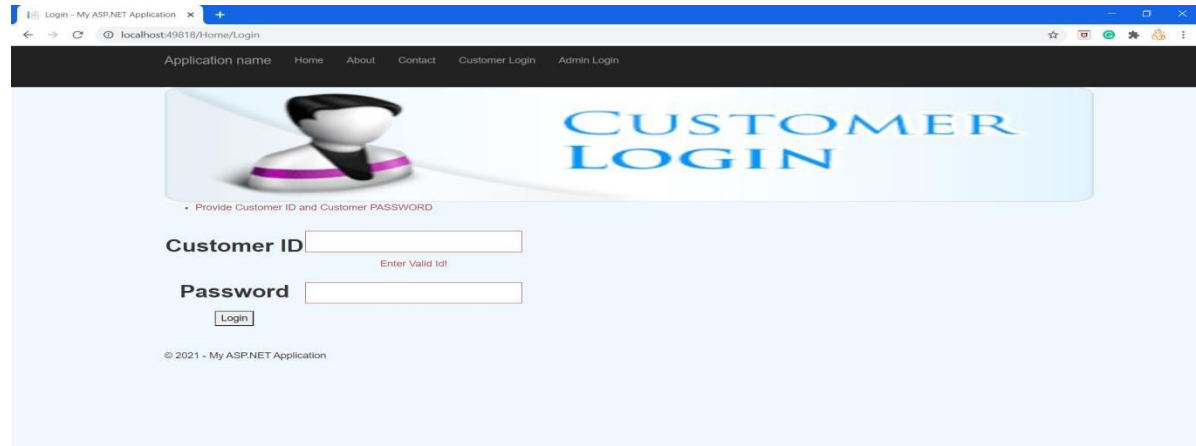
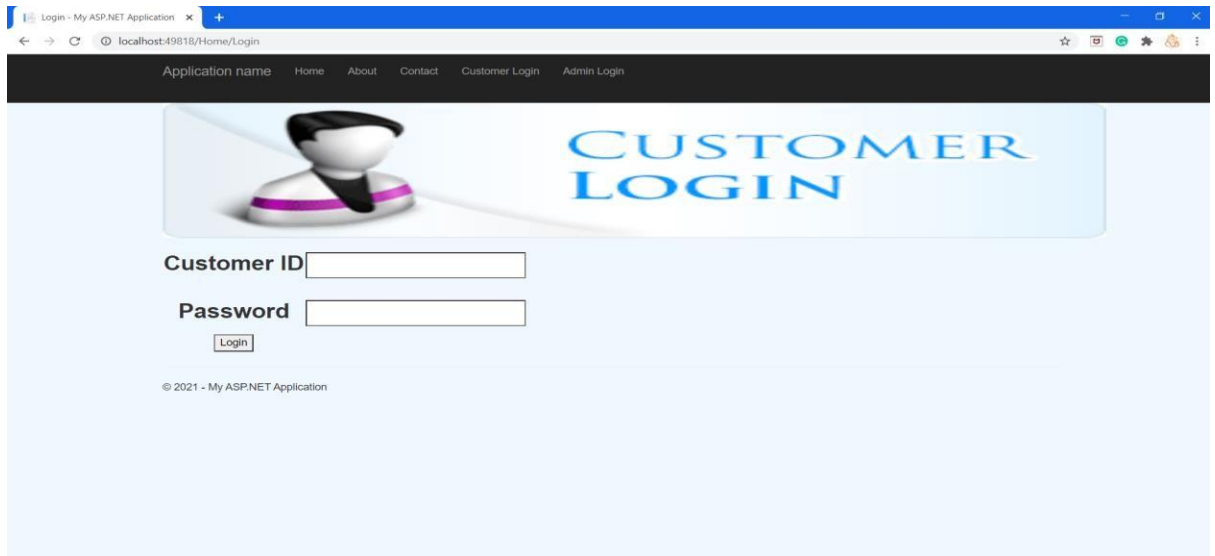


Fig: 5.6 Customer login Test case 6

6. OUTPUT SCREENS

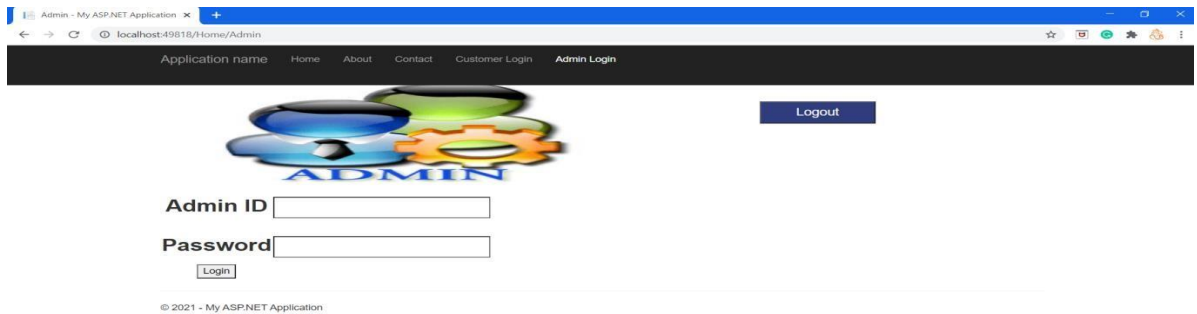
6.1 LOGIN PAGE



The screenshot shows a web browser window with the title "Login - My ASP.NET Application". The address bar shows "localhost:49816/Home/Login". The page has a dark navigation bar with links: "Application name", "Home", "About", "Contact", "Customer Login", and "Admin Login". The main content area has a light blue background. At the top, there is a banner with a 3D scanner icon and the text "CUSTOMER LOGIN". Below the banner, there are two input fields: "Customer ID" and "Password". A "Login" button is positioned below the "Password" field. At the bottom, there is a copyright notice: "© 2021 - My ASP.NET Application".

Fig: 6.1 Login Page

6.2 ADMIN PAGE



The screenshot shows a web browser window with the title "Admin - My ASP.NET Application". The address bar shows "localhost:49816/Home/Admin". The page has a dark navigation bar with links: "Application name", "Home", "About", "Contact", "Customer Login", and "Admin Login". The main content area has a light blue background. At the top, there is a banner with a 3D gear icon and the text "ADMIN". Below the banner, there are two input fields: "Admin ID" and "Password". A "Login" button is positioned below the "Password" field. A "Logout" button is located in the top right corner. At the bottom, there is a copyright notice: "© 2021 - My ASP.NET Application".

Fig: 6.2 Admin Page

6.3 ADMIN CONTROL PAGE

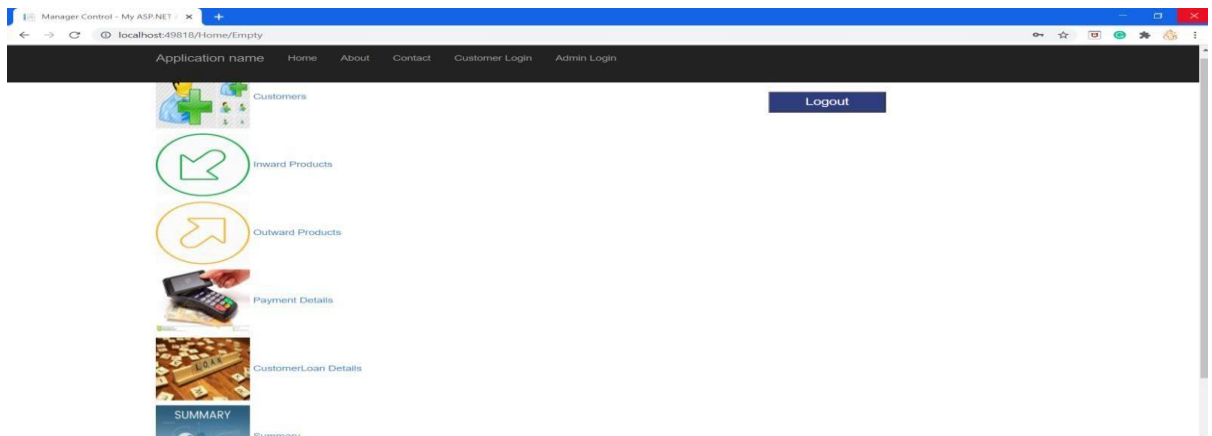


Fig: 6.3 Admin control page

6.4 CUSTOMER ENTRY FORM FOR ADMIN

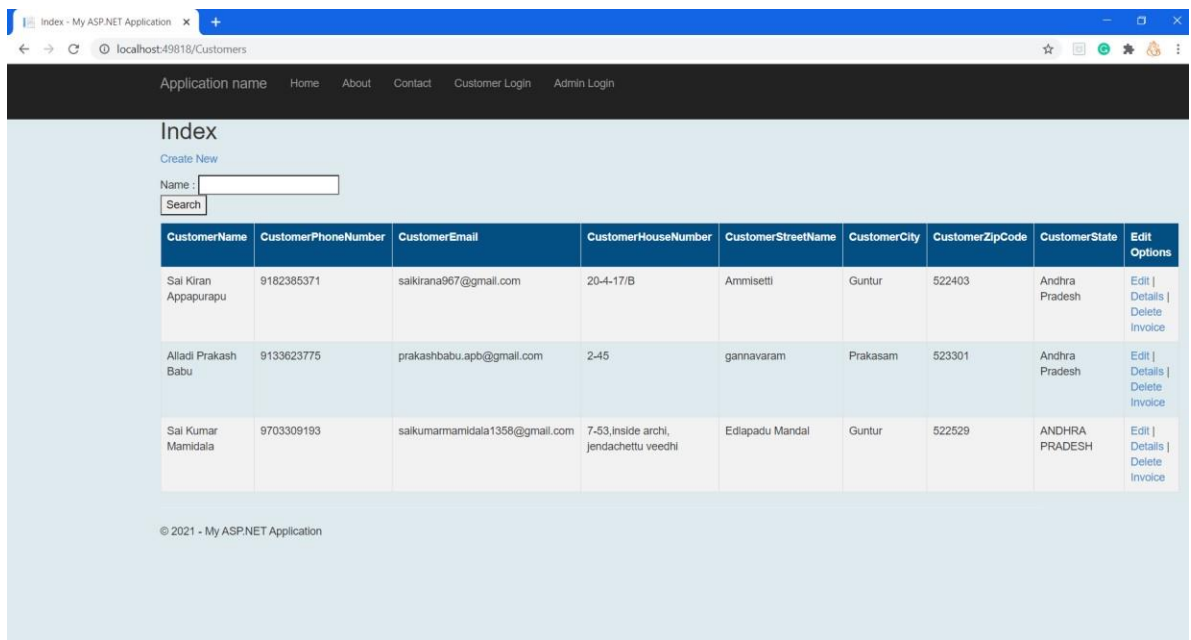
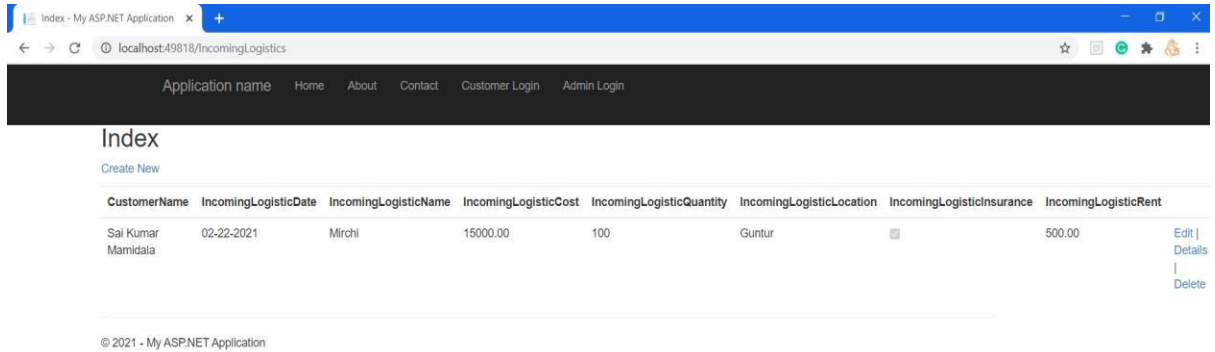


Fig: 6.4 Customer entry form for admin

6.5 INCOMING PRODUCT ENTRY FORM FOR ADMIN



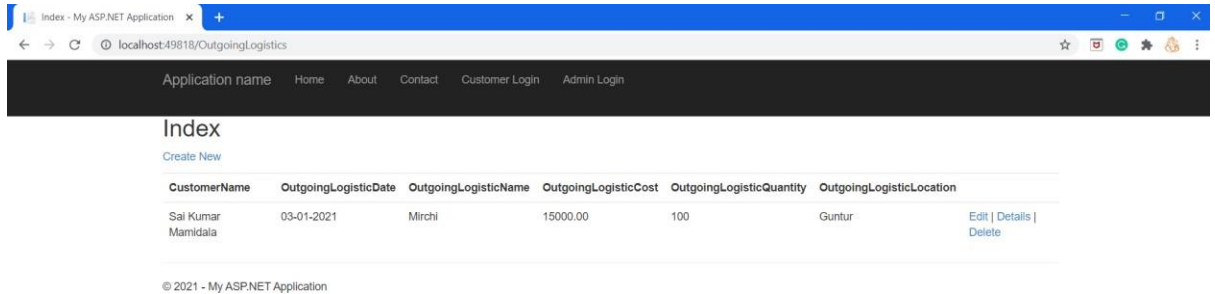
The screenshot shows a web browser window with the URL `localhost:49818/IncomingLogistics`. The page has a dark navigation bar with links: Application name, Home, About, Contact, Customer Login, and Admin Login. Below the navigation bar, the page title is "Index". There is a "Create New" link. A table displays incoming logistics data. The table has columns: CustomerName, IncomingLogisticDate, IncomingLogisticName, IncomingLogisticCost, IncomingLogisticQuantity, IncomingLogisticLocation, IncomingLogisticInsurance, IncomingLogisticRent, and a column for actions (Edit, Details, Delete). The table contains one row of data for Sai Kumar Mamidala.

CustomerName	IncomingLogisticDate	IncomingLogisticName	IncomingLogisticCost	IncomingLogisticQuantity	IncomingLogisticLocation	IncomingLogisticInsurance	IncomingLogisticRent	
Sai Kumar Mamidala	02-22-2021	Mirchi	15000.00	100	Guntur	☒	500.00	Edit Details Delete

© 2021 - My ASP.NET Application

Fig: 6.5 Incoming product entry form for admin

6.6 OUTGOING PRODUCT ENTRY FORM FOR ADMIN



The screenshot shows a web browser window with the URL `localhost:49818/OutgoingLogistics`. The page has a dark navigation bar with links: Application name, Home, About, Contact, Customer Login, and Admin Login. Below the navigation bar, the page title is "Index". There is a "Create New" link. A table displays outgoing logistics data. The table has columns: CustomerName, OutgoingLogisticDate, OutgoingLogisticName, OutgoingLogisticCost, OutgoingLogisticQuantity, OutgoingLogisticLocation, and a column for actions (Edit, Details, Delete). The table contains one row of data for Sai Kumar Mamidala.

CustomerName	OutgoingLogisticDate	OutgoingLogisticName	OutgoingLogisticCost	OutgoingLogisticQuantity	OutgoingLogisticLocation	
Sai Kumar Mamidala	03-01-2021	Mirchi	15000.00	100	Guntur	Edit Details Delete

© 2021 - My ASP.NET Application

Fig: 6.6 Outgoing product entry form for admin

6.7 PAYMENT DETAILS ENTRY FORM FOR ADMIN

Application name Home About Contact Customer Login Admin Login

Index
[Create New](#)

CustomerName	PaymentType	PaymentAmount	PaymentDate	PaymentTime	PaymentAdvance	PaymentDue	Edit Options
Sai Kumar Mamidala	CardLess	15000.00	03-01-2021	22:47	0.00	0.00	Edit Details Delete

© 2021 - My ASP.NET Application

Fig: 6.7 Payment details entry form for admin

6.8 CUSTOMER LOAN ENTRY FORM FOR ADMIN

Application name Home About Contact Customer Login Admin Login

Index
[Create New](#)

IncomingLogisticName	CustomerLoanDate	CustomerLoanAmount	CustomerLoanBankName	CustomerAccountNumber	CustomerLoanBankIFSCCode	Edit Options
Mirchi	02-22-2021	15000.00	SBI	20493247577	SBIN0005689	Edit Details Delete

© 2021 - My ASP.NET Application

Fig: 6.8 Customer loan entry form for admin

6.9 SUMMARY FORM FOR ADMIN

Summary - My ASP.NET Application
localhost:49818/Home/Summary
Application name Home About Contact Customer Login Admin Login

Summary

Customer Details

Customer ID	Customer Name	Customer Phone Number	Customer Email	Customer House Number	Customer Street Name	Customer City	Customer Zip Code	Customer State
2	Sai Kiran Appapurapu	9182385371	salkirana967@gmail.com	20-4-17/B	Ammisettil	Guntur	522403	Andhra Pradesh
4	Alladi Prakash Babu	9133623775	prakashbabu.apb@gmail.com	2-45	gannavaram	Prakasam	523301	Andhra Pradesh
1007	Sai Kumar Mamidala	9703309193	saikumarmamidala1358@gmail.com	7-53,inside archi, jendachettu veedhi	Edlapadu Mandal	Guntur	522529	ANDHRA PRADESH

OutgoingLogistic Details

Customer ID	OutgoingLogistic ID	OutgoingLogistic Date	OutgoingLogistic Name	OutgoingLogistic Cost	OutgoingLogistic Quantity	OutgoingLogistic Location
1007	35	01-03-2021 00:00:00	Mirchi	15000.0000	100	Guntur

Payment Details

Customer ID	Payment ID	Payment Type	Payment Amount	Payment Date	Payment Time
1007	5	CardLess	15000.0000	01-03-2021 00:00:00	22:47:00

© 2021 - My ASP.NET Application

Fig: 6.9 Summary form for admin

7. CONCLUSION

- The project entitled Cold Storage Management System was completed successfully. The system has been developed with much care and free of errors and at the same time it is efficient and less time consuming. The purpose of this project was to develop a web application customer can easily can get his profile and invoice and Admin(Manager) can maintain all details of products ,customer and payments. This application can reduce paper work. This project helped us in gaining valuable information and practical knowledge on several topics like designing web pages using html & CSS, usage of responsive templates, and management of database using SQL.
- The entire system is secured. Also the project helped us understanding about the development phases of a project and software development life cycle. We learned how to test different features of a project. This project has given us great satisfaction in having designed an application which can be implemented to any nearby cold storages. There is a scope for further development in our project to a great extend. A number of features can be added to this system in future like providing stock details of customers and maintaining transaction communication between manager and customer through messages and emails. Another feature we wished to implement was providing classes for customers so that different offers can be given to each class. System may keep track of history of inward products, outward products, payment of each customer. These features could have implemented unless the time did not limited us.

8. FUTURE ENHANCEMENT

- We would like to improve in Customer Interface.
- We would like to improve admin interface.
- Efficient security features provides data privacy and data integrity.
- More functionality can be added depending upon user requirements and specifications

9. BIBLIOGRAPHY

[1] JavaScript Enlightenment, Cody Lindley-First Edition, based on JavaScript 1.5, ECMA-262, Edition - 2

[2] Complete CSS Guide ,Maxine Sherrin and John Allsopp-O'Reilly Media; September 2012

[3] Online references:

- <http://www.w3schools.com/html/default.asp>
- <http://www.w3schools.com/css/default.asp>
- <http://www.w3schools.com/js/default.asp>
- <https://dotnet.microsoft.com/>
- <https://dotnetspider.com/>
- <https://www.google.com/>